6.3 Software

Overview

The code for the ESP32-C5 is written in C++, making use of header and C++ file for each class. The code writing in vscode and is structured in folders for clarity. The extension for platformIO is used to compile the code for this specific board. Some of the implemented behaviour was inspired by the code for the MAMRI v5, the PnuematicCyllinder library is written by V. Groenhuis and the KinematicsController is written by S. Benders. However, the majority of the code was written from scratch to improve the structure, and create separation of concerns. Where if possible the hardware specific code is abstracted so a simpler functions are created for the higher level code to interface with. Below this the simplified class diagram is given, and is explained.

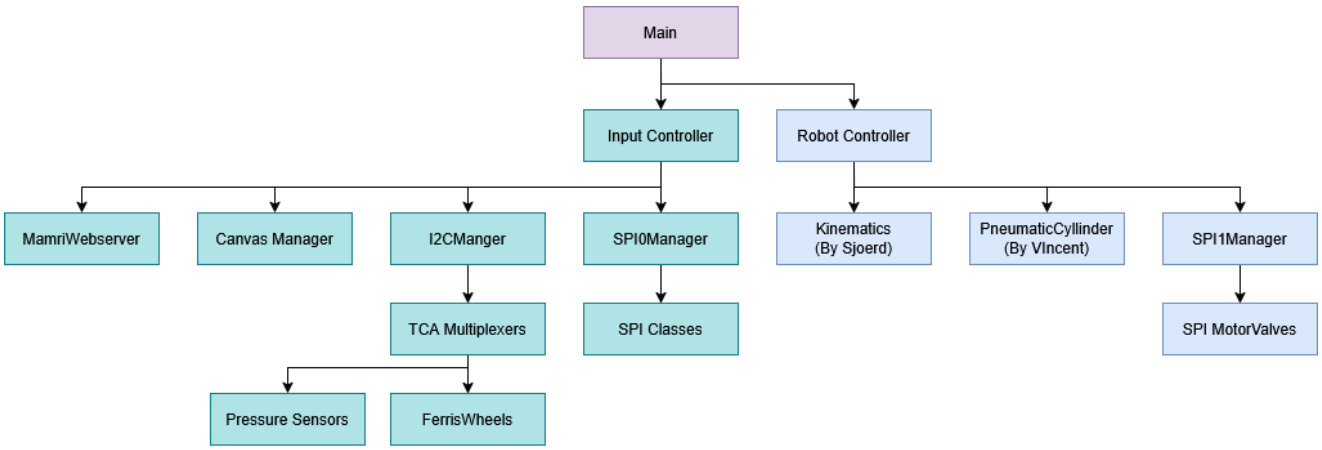

Fig. 6.11. Simplified Class Diagram.

The main file initializes the input and robot controllers, sets the pinmodes, sets up the async webserver and starts the interrupt timer to service the motors.

The input controller manages all types of sensor or user input and determines the higher level behaviour of the controller, it maps the incoming input to a behaviour. The SPI0Manager provides simple get and set functions, to abstract away underlying CS functionality. For example the InputController reads out the horizontal buttons with the SPI manager, then via the RobotController reference it can increment the joint steps based on the button presses. Similar to the SPI Manager is the I2C manager. This provides a get angle function, and can create a dynamic number of TCA and FerrisWheel objects. The Async Webserver runs in the background and is serviced when a client connects to an endpoint.

The robot controller manages the stepper motors, the current selected robot via the robot configurations. It has a SPI1Manager child that handle the motor valves. The PneumaticStepper class, written by V. Groenhuis holds a target position and a current position, which is used as a model for each stepper motor. It handles the cylinder states, which are read out and written to the main valve. Based on the robot config a robot can have kinematics. When this is required a kinematics object with the correct settings is created and can be used later to do FK or IK. The kinematics class is written by S. Benders.

Robot Controller

The previous MAMRI, used a class called 'Mamri'. The Robot controller class mimic this behavior. So does it also make use of the Pneumatic Cylinder class to keep track of the state of the stepper motors and to set target positions. In the setup of main a timer interrupt is started. This works based on a set frequency and when the service function is called the work function of the pneumatic steppers is called, as well as the setValves() function of SPI1Manager, which actually opens the correct valves based on the internal state of each stepper object.

What makes this different is the implementation of dynamic switching of robot configurations, and the structure for this controller and the kinematics controller. This is a child of the robot controller and can be initialized based on whether the selected robot config requires kinematics to be used.

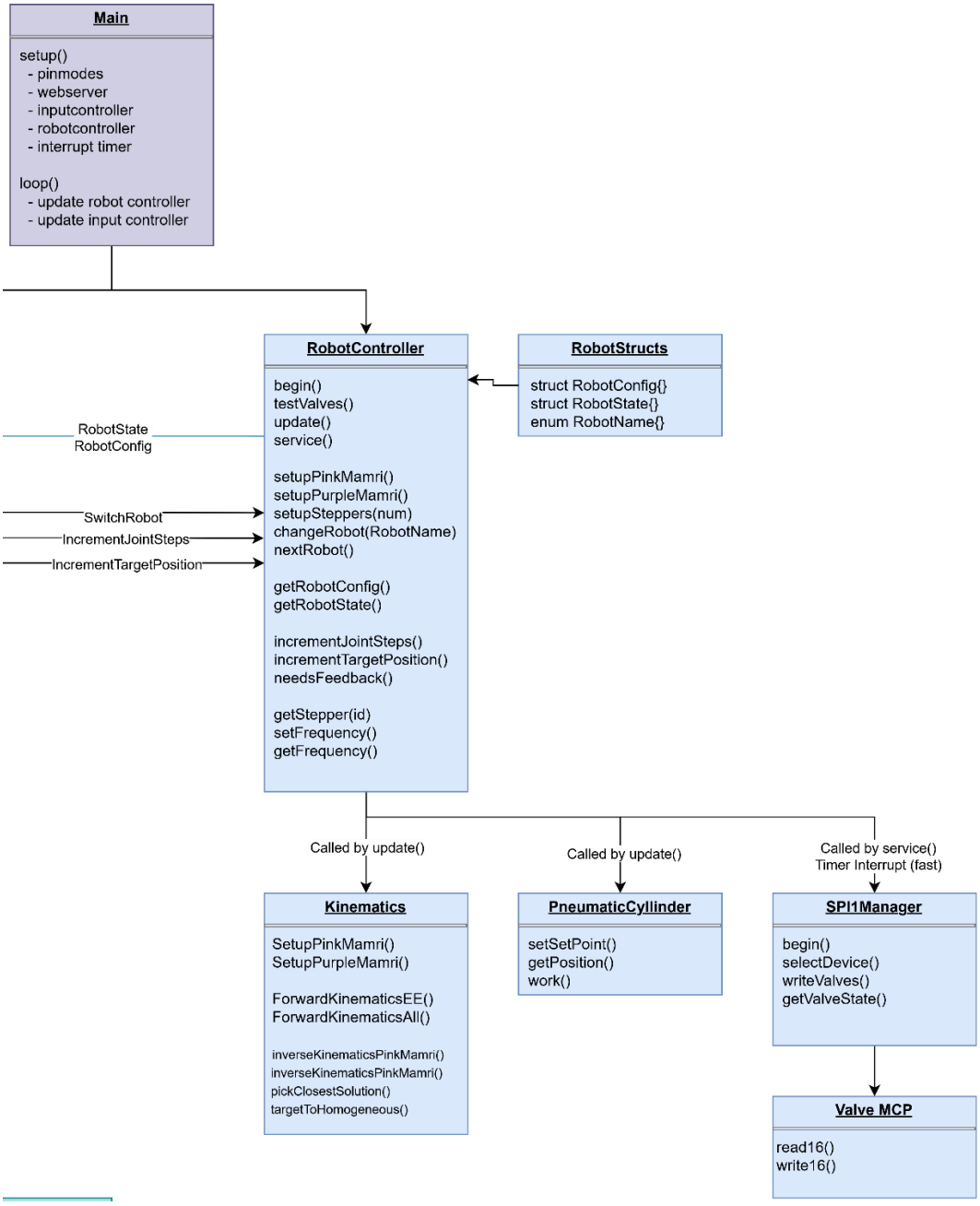

Fig. 6.12 Robot Controller side of class diagram. Showing the RobotController, robotStructs, kinematicsController, PneumaticCyllinder, SPI1Manager and Valve MCP.

In Fig 6.12 the most important public functions for each class are visible. The RobotController is initialized in main, and a default robot is selected. Then in the main loop the RobotController is updated. Here it updates the setpoints of each stepper motor, and the current robotState. Besides the main update loop there are several getter and setter functions for the most important variables, like stepper objects, motor frequencies, robot config and robot state.

The last two are structs as this makes it easy to define a global structure for several variables in one. So does the robot configuration store a robotName, number of steppers, degrees

of freedom, which vales are inverted, whether there is positional feedback, the number of FerrisWheel and whether there is a kinematics object for this robot. The robot state stores a list of jointSteps and targetJoint steps. And also lists of currentPositions and targetPositions. (x, y, z, rotX, rotY, rotZ)

When the target positions or actual positions have changed the RobotController can set a flag high to notify the main loop that positional feedback is needed. If true the InputController is called to fetch new Ferris Wheel data.

The kinematicsController has specific constructors for different robots, this defines the twists, links and other kinematics of the specific robot. Using this class a general solution for the inverse kinematics can be found. From the multiple solutions a specific solution can be selected, and with those joint steps the robots joint target can be set. This initialization of this class based on the robot type is implemented, but the class is not called to actually perform forward or inverse kinematics.

Input Controller

The input controller is used for the higher level behavior of the Grace MAMRI Controller. By using the SPI and I2C Managers, as well as the webserver user input, or sensor input can be gathered and processed, without having to deal with the hardware specific code. So these classes make use of abstraction. Most of the behavior is captured in the update function, other functions are used to set or get different internal variables. Using the buttons jointPositions or targetPositions can be incremented in the RobotController. Based on the RobotConfig the correct number of buttons is processed. The potentiometers are used to set the motor frequency and the InputMode. Using the vertical buttons a user can switch robot types by calling the nextRobot() function in the RobotController. But also, restart the Ferris Wheels when a ferris wheel error occurred, add steppers in stepper test mode or clear the target positions of a robot to 'home' it.
Below the full input side of the class diagram is shown.

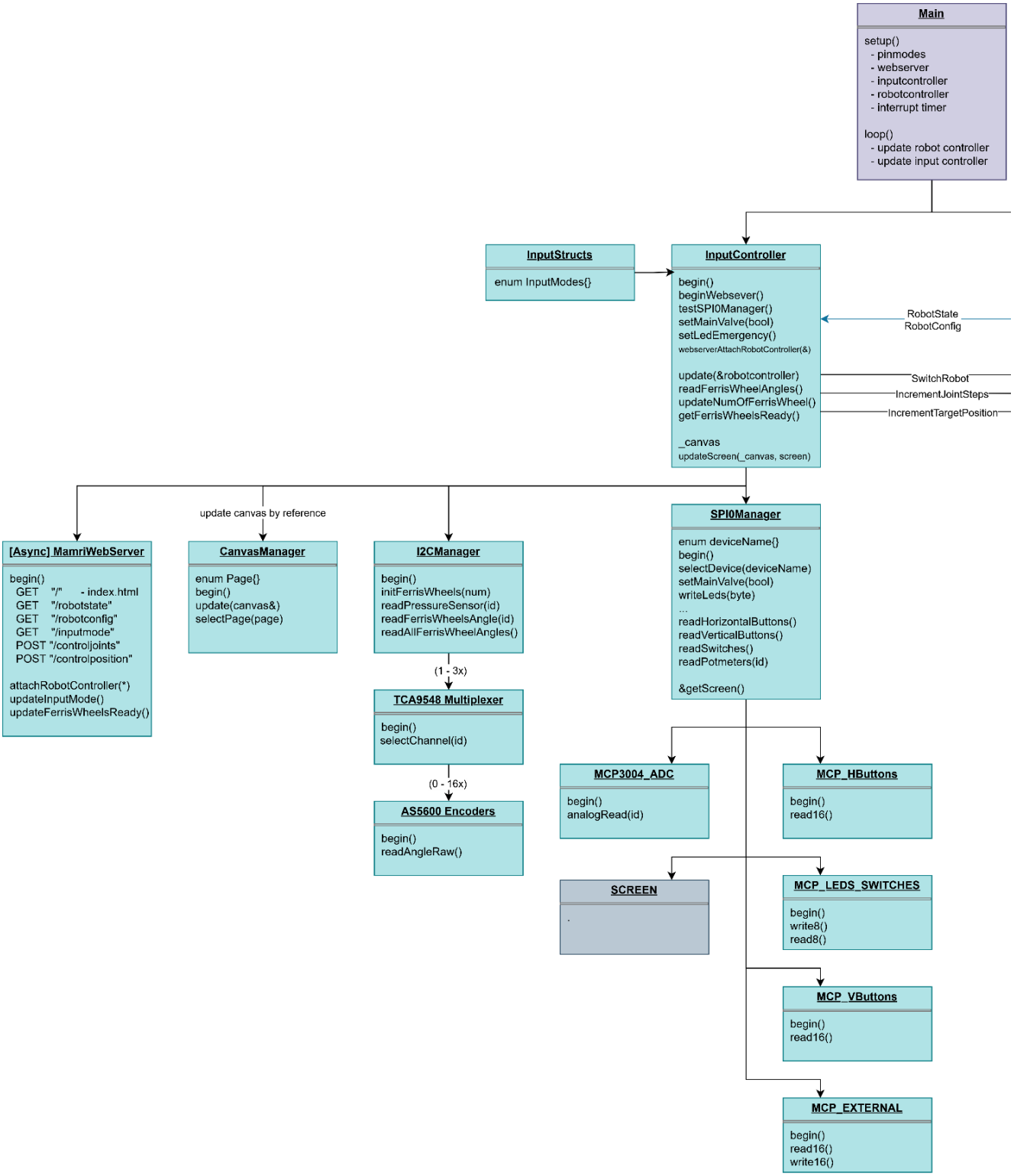

Fig. 6.13. Input Side of Class Diagram. Input Controller which defines higher level behavior. SPI0 Manager as a wrapper for chip selection. With MCP23S17 objects, MCP3004 ADC and TFT Screen. I2C Manager handles a variable number of TCA chips, and AS5600 Ferris Wheel sensors. The canvas manger draw to the canvas. MamriWebserver starts an Async webserver, with references to Input en RobotController for data.

SPI0 Manager and Chip Select

Because of the hardware limitations of the ESP32-C5, which has fewer GPIO pins, a work around is created using a Demultiplexer (Demux). Using 4 bits we can pull one of 16 pins low. For an SPI transfer the Chipselect (CS) pin has to be pulled low in order for that specific device to start listening to the data that is being transferred over the MOSI line. This is the Master Out, Slave In channel. Similarly when the controller wants to listen to an SPI device over the MISO line, Master In, Slave Out, line the CS also has to be low. The SCK, or clock line is used to time each bit, so both devices know when 1 bit starts.

The Mamri controller uses 3 different kinds of devices. The MCP23S17 GPIO expander, which has 2 banks with each 8 GPIO pins. These can be configured for input or output, but also set for a pullup. Then read and write function in the library can be called to readout these devices. The second device is the MCP3004 analog to digital converter (ADC), which is used to read out the two potmeters. Lastly the Mamri Controller has a thin film transistor screen (TFT). This also communicates over the SPI0 bus.

One difficulty is that these libraries usually expect a CS pin in the constructor. Then the library manages the pinmode, and digitalwrite. This is something we want to prevent as these pins are not available. Therefore the MCP23S17 and MCP3004 libraries are edited so they do not rely on a CS pin. The library for the screen had an option to set CS to -1, with this enabled the library did not use any GPIO pin.

When doing an SPI transfer it is up to the SPI0Manager to make sure the correct chip is selected. This is done through a wrapper function which can be called by its parent, the InputController. For external SPI devices a new device can be added to the deviceName enumerator, and a corresponding wrapper function can be created to communicate with the device.

SPI1 Manager and Interrupt Timer

The Mamri v5 Controller also has two SPI busses. This is done for a specific reason, as the timing for the actuation of the robots stepper motors is important. For this purpose an interrupt timer is started in the main file. This timer will run a function at a specified interval, which frequency can be set in the system parameters file. This timer causes any code to stop running, so the webserver, SPI0 and I2C transfers will all freeze for a short time. It is important to make the function that is ran on this interrupt be very quick. Only the work function of the stepper motors, and a transfer on SPI1 will run during this interrupt.

If the motor valves were connected on the same SPI bus as the other devices, it could be possible that during a transfer to any of the SPI0 devices, a different transfer could start,

corrupting the previous transfer. But when the motor valves are connected to a separate bus the previous transfer can continue. This works because of the SCL, or clock line which also freeze. Similarly the I2C clock freezes and can continue after the interrupt.

To start a transfer with an SPI device it is important to toggle this CS pin. Therefore, in the select device function, NO_DEVICE is selected and quickly thereafter the wanted device is selected. For this purpose the 0 pin of the CD4515 demux chip is left floating.

The manager for SPI0 controls 6 devices, whereas the SPI1 manager only controls 1 device. Still it was chosen to create this wrapper class for consistency. If in the future SPI bus 1 is extended too, the correct structure is in place. But is will also help developers of the MAMRI systems to better understand the CS behavior.

I2C Manager and Ferris Wheels

To add more devices to the I2C bus without having to set a unique address for each device a TCA chip can be used. These are multiplexers and select one channel. The I2C manager makes sure the correct TCA chip is addressed, as these do have a unique ID. Then a command is send to select a channel. And after this a command can be sent to the sensor on the other side of the multiplexer.

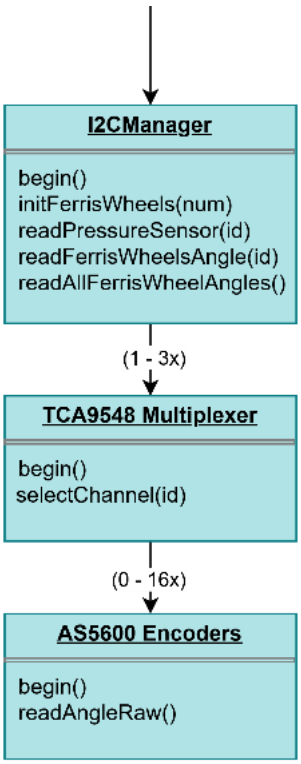

Fig. 6.14 Class I2C Manager. During runtime a variable number of Ferris Wheels can be reinitialized based on the selected robot. (0-16) Child of the InputController.

Currently, other students are developing the MAMRI systems which require absolute positional feedback. Using AS5600 Magnetic encoder sensors a precise angle can be read out. A challenge with these sensors is that they have to be integrated into the MAMRI controller. The difficult part is to have the user connect the correct amount of ferris wheels that a robot needs. This is stored in the robot configuration, and the initFerrisWheels function check if that same number of FerrisWheels is connected. This is done by making use of dynamic initialization of AS5600 objects (ferris wheel objects) based on the value in the robot config. With this the user can switch robots in runtime, or connect new ferris wheel in runtime, without having to change the code. This removes the need for changing the code for a different robot. When the Ferris Wheels do not begin properly an error is returned and indicated to the user via one of the Red LEDs on the MAMRI Controller. When the user presses the restart FerrisWheels button, the I2C manager will try to reinitialize the ferris wheels, and remove the error if successful.

The I2C manager can also readout the pressure with one of the two pressure sensors. This could be used to double check the pressure in the system and can help to get better performance for the stepper motors. This was not implemented in the current version due to time constraints.

Canvas Manager

Was created but not integrated fully. It uses the canvas object owned by the Input Controller to do draw functions separately from the Input Controller to separate concerns. Then this canvas object can be passed to the SPI0Manager to actually draw the canvas to the screen. This was not implemented due to time constraints. Limitations in working memory prevented the use of a 16bit color canvas. A version with 1 bit color, black and white was tested but drawing to the screen was not realized.

Webserver

The asynchronous webserver is started early in the main setup. This is done to ensure that enough RAM is available for the RTOS task that is needed for this webserver to operate properly. The advantage of an asynchronous webserver is that it does not require an update call, it will be updated whenever a client sends a request on an specific endpoint.

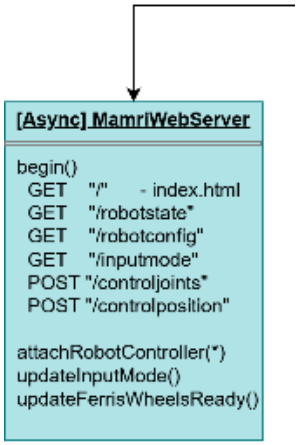

Fig. 6.15 Class Diagram of Webserver. Showing all endpoints and functions. Child of the InputController.

An endpoint is one of the URLs that are configured by the webserver. This server uses two types, several GET endpoints and two POST endpoints. The difference between the two is, that for a GET requests the user requests some data, like a webpage or JSON data. Whereas a POST request is used more often as a way to upload data from the user to the server. In this case control for either joints or target position can be done through this webserver.

The webserver has access to the RobotController via a pointer that is passed to the class after the RobotController is initialized. It can call the public get functions of this class. Such as, getRobotConfig and getRobotState. This class has access to the InputController via a reference which is pass to the update function of the webserver class. Then the InputController, which is the parent of the webserver can simply pass a reference to itself to the child. This was done differently to prevent the use of pointers. But also because it was not possible to pass the RobotController as a reference because the webserver is initialized before the robot controller. This was required due to the restriction on working memory.

Webpage

With the webserver running on the ESP, a client can connect to the IP address on the same Wi-Fi network. On university a Wi-Fi network called iotroam is hosted for use cases like that of this project. By reserving a static IP address through a self service portal the ESP can connect to this network. When the user is connected to eduroam they can also connect to the ESP, which is presumable forwarded to the eduroam network. Index.html is served on the root endpoint "/".

One advantage of using a webpage is that all the computation for the visualization is done by the client. Whereas if this is done by the MAMRI controller it would take more computational power to render a 3D scene on the TFT Screen, possibly slowing down the controller in the process.

By making use of the JavaScript library called THREE.js, it is possible to create 3D scenes with different objects, lights and a camera. This webpage regularly fetches the state of the robot and its config by polling the GET endpoints discussed before. This is displayed on the right hand side of the viewing window. And includes: the robot state with all the positions, the robot config, motor frequency and the current selected input mode.

On the left hand side the correct number of sliders is displayed. So if a robot has 6 stepper motors, also 6 sliders will be drawn. When changes are made to the sliders control commands are send to the POST endpoints and will update the values in the robot controller via setter functions. These are available to the webserver via the pointer of the RobotController.

In the center the robot render can be seen. The robot is rendered based on a configuration in the index.html file, and is not dependent on the actual config stored in the RobotController. This was not implemented due to time constraints. The twists of each joint are defined and each joint correctly rotates around a specific axis. For the purple Mamri this means that the base rotates around the vertical z axis, whereas the next 3 joints rotate around the x axis. (The 5th joint was not added, but should rotate on the z axis again.) The current joint steps are mapped to the angle of each joint. For an estimated radians per step was used, and defined for each joint. For a real implementation this should be more accurate, and preferably based on the kinematics controller settings.

Lastly, a slider is provided to change the viewing angle of the camera. With it the user can rotate around the robot to take a better view.

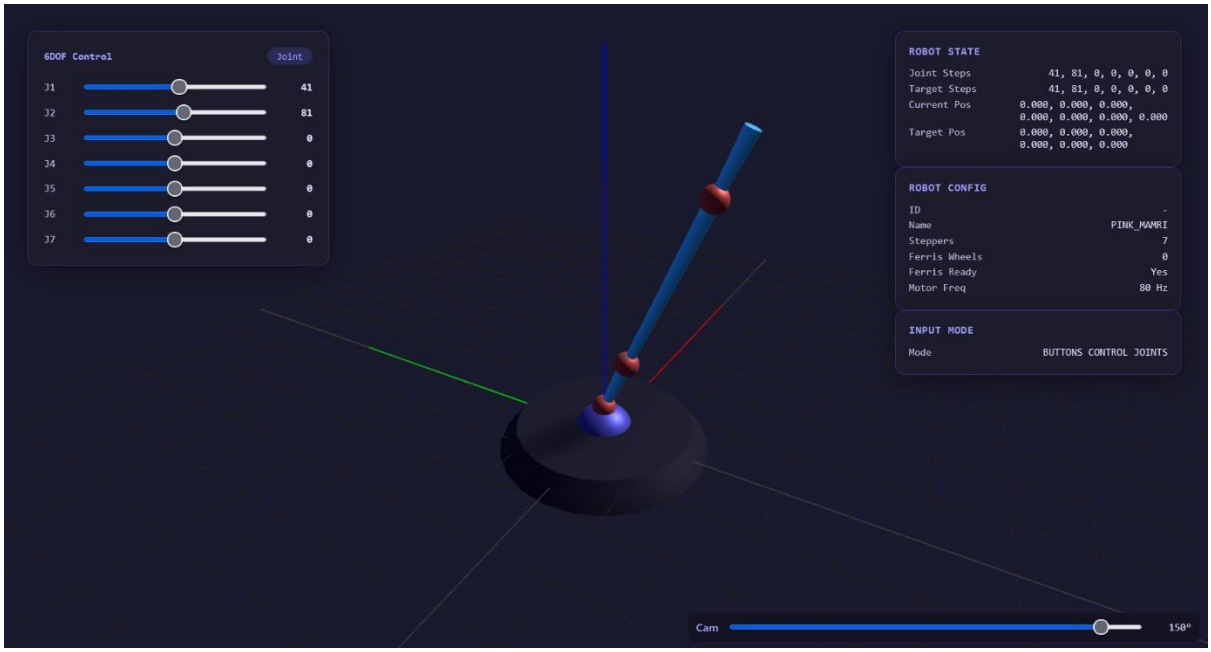

Fig. 6.16 Index.html. Showing a robot visualization of the Purple Mamri, sliders for joint control and robot information.

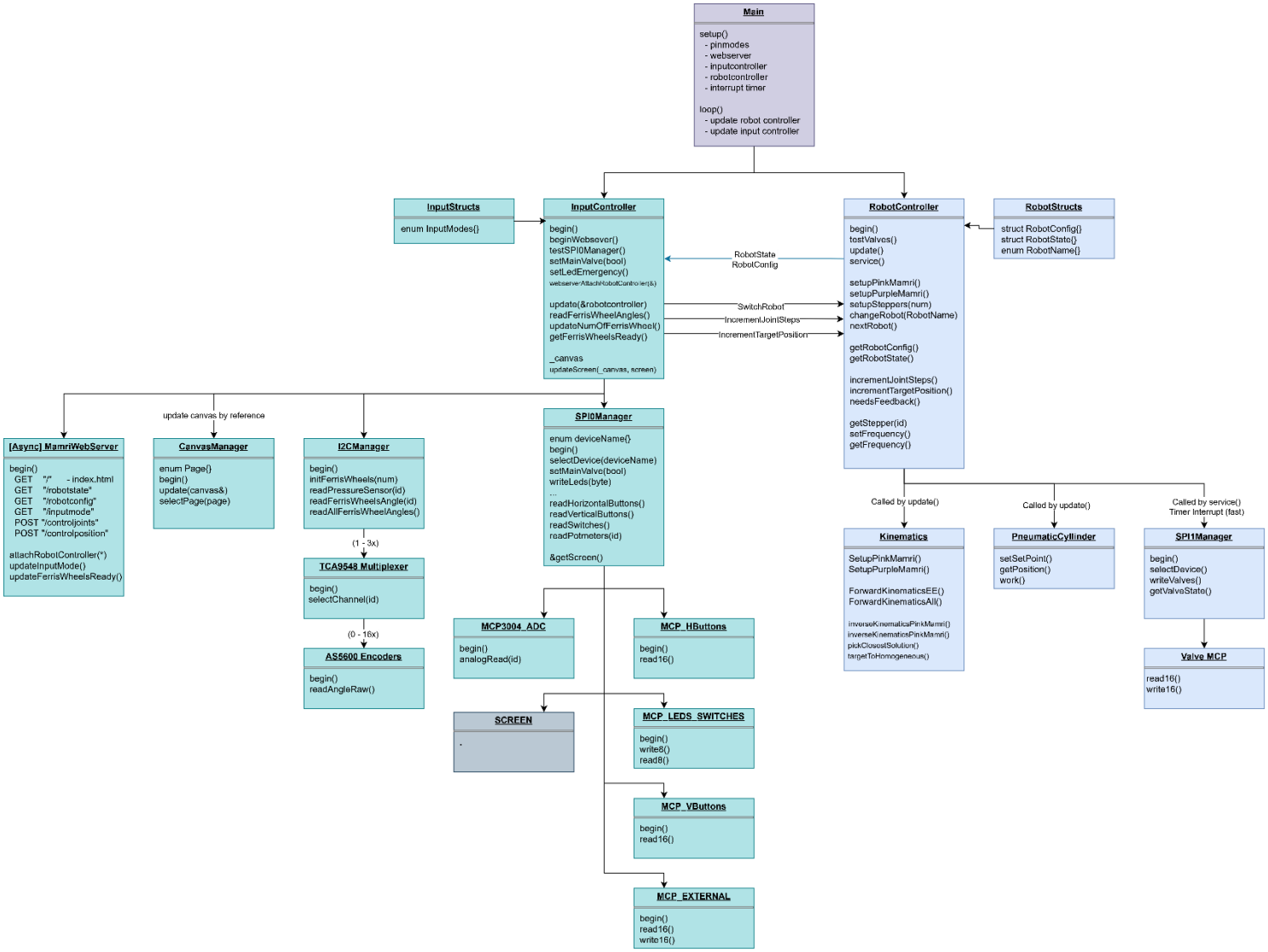

Fig. 6.17 Full class diagram